SKILL - 8

Name: Hansika Reddy Gunapati

Rollno: 2520030237

Section: 8

Aim

To detect variable references in command input, expand their values, handle undefined variables, update parsed tokens, support nested variables, and test the expansion logic. Also, to identify built-in commands, create a dispatch table, execute in-process commands, handle invalid commands, maintain shell state, and test the command dispatch logic.

Objective

- To detect variable references in command input.
- To expand variables to their stored values.
- To handle undefined variables correctly.
- To update tokens after variable expansion.
- To support nested variable references.
- To test the variable expansion logic.
- To identify built-in commands.
- To create a command dispatch table.
- To execute built-in commands in-process.
- To handle invalid commands.
- To maintain program state while executing built-ins.
- To test the command dispatch logic.

Problem Statement

Design and implement a command-processing system that identifies variable references in input and replaces them with their corresponding values. The system should handle undefined and nested variables and update the resulting command tokens. It should also identify built-in commands through a dispatch table, execute them within the current process, report invalid commands, maintain state, and verify the complete dispatch logic.

Theory

Variable expansion is an important stage in command processing. A command may contain references to variables whose values must be substituted before the command is executed.

1. Variable References

A variable reference is a symbol or name used to represent a stored value. During expansion, the parser identifies the variable reference and replaces it with the corresponding value.

2. Variable Expansion

Variable expansion converts a token containing a variable reference into a token containing its value. For example, if USERNAME stores Hansika, an input such as $USERNAME can be expanded to Hansika.

3. Undefined Variables

If a referenced variable does not exist, the program must handle the situation consistently. The implementation can report that the variable is undefined or expand it according to the selected shell rule.

4. Token Updating

After a variable is expanded, the token must be updated before the command is dispatched. The updated token array is then used for subsequent processing.

5. Nested Variables

Nested variables occur when the value of one variable contains a reference to another variable. The expansion logic must continue resolving references until the required final value is obtained or a termination condition is reached.

6. Built-in Commands

Built-in commands are commands handled directly by the command interpreter. They are executed in the current process because some of them need to modify shell state, such as changing directories or updating stored variables.

7. Dispatch Table

A dispatch table maps the name of a built-in command to the function that implements it. Instead of checking every command through a long sequence of conditions, the program can look up the command and call the associated function.

8. In-Process Execution

A built-in command is executed within the current process. This is different from launching an external program in a child process because changes made by the built-in must remain in the current command interpreter.

9. Invalid Commands

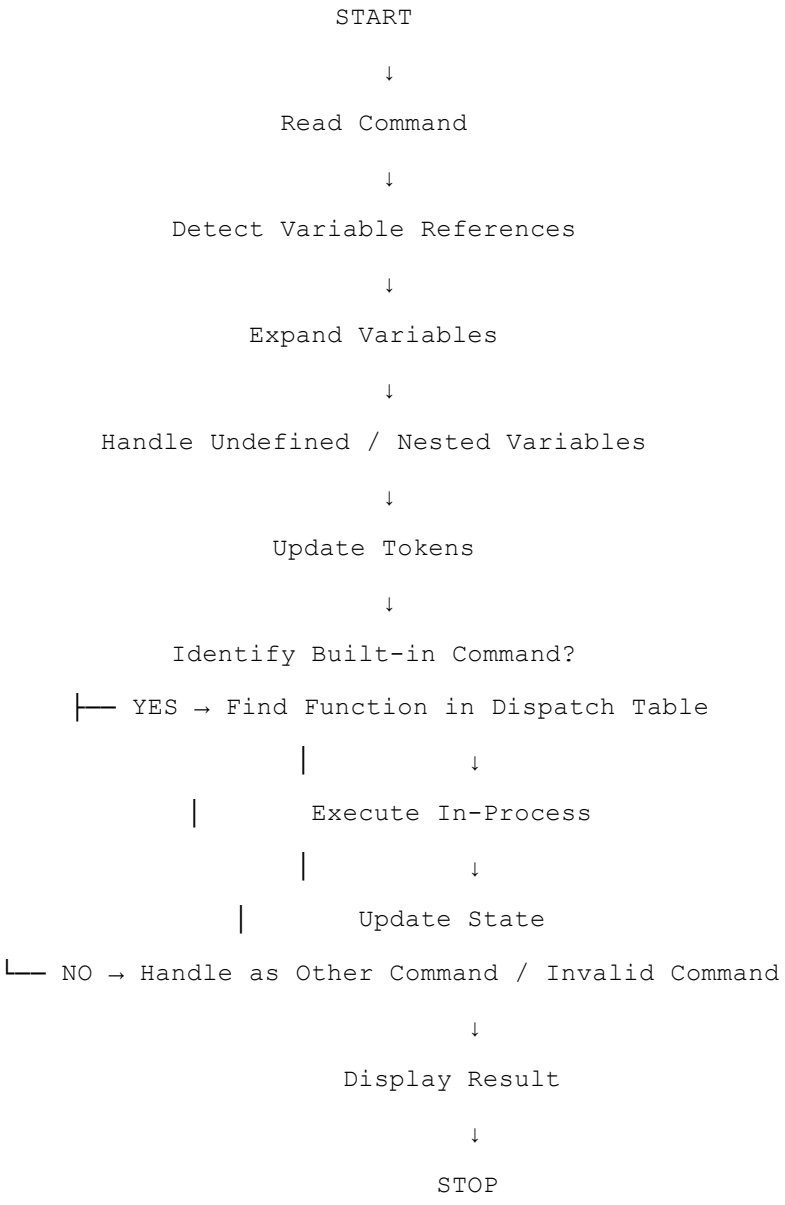
When a command does not match any registered built-in and is not handled as an external command, the program should display an appropriate invalid-command message.

10. State Maintenance

The command interpreter maintains state such as the current directory and stored variables. Built-in commands are able to update this state directly because they execute in the current process.

11. Control Flow

The control flow of the combined variable expansion and built-in dispatch process is represented below:



12. Algorithm

1. Start the program.
2. Initialize a small variable table with sample variable names and values.
3. Read a command string.
4. Split the command into tokens.
5. Check each token for a variable reference beginning with a variable marker.
6. Extract the variable name and search for its stored value.
7. Replace the reference with the stored value when found.
8. Handle undefined variables according to the program rule.
9. Repeat expansion when a variable value contains another variable reference.
10. Update the token with the expanded value.
11. Check whether the first token matches a built-in command.
12. Search the built-in dispatch table for the corresponding function.
13. If a built-in is found, execute it in the current process.
14. If the command is invalid, display an appropriate error message.
15. Maintain any state changes produced by built-in commands.
16. Display the result and stop.

13. Data Used

A variable table stores variable names and their values. A token array stores the command after it has been split. A dispatch table stores built-in command names and their associated functions. Integer values are used for token counts and lookup indexes.

14. Operations Used

- Variable reference detection.
- String comparison and variable lookup.
- Token replacement.
- Nested variable expansion.
- Built-in command lookup.
- Function-pointer based dispatch.
- In-process command execution.
- State update.
- Invalid-command handling.

15. Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 100

typedef void (*CommandFunc)(char *);

typedef struct {
    char name[20];
    char value[100];
} Variable;

Variable vars[] = {
    {"NAME", "Hansika"},
    {"CITY", "Hyderabad"},
    {"MESSAGE", "$NAME is from $CITY"}
};

int varCount = 3;

const char *getValue(const char *name) {
    for (int i = 0; i < varCount; i++)
        if (strcmp(vars[i].name, name) == 0)
            return vars[i].value;
    return NULL;
}

void expand(char *input, char *output) {
    int i = 0, j = 0;

    while (input[i] != '\0') {
        if (input[i] == '$') {
            char name[30];
            int k = 0;
            i++;

            while (input[i] != '\0' &&
                   ((input[i] >= 'A' && input[i] <= 'Z') ||
                    (input[i] >= 'a' && input[i] <= 'z') ||
                    (input[i] >= '0' && input[i] <= '9') ||
                    input[i] == '_')) {
                name[k++] = input[i++];
            }

            name[k] = '\0';

            const char *value = getValue(name);

            if (value != NULL) {
                char temp[200];
                expand((char *)value, temp);
```

```
                strcpy(&output[j], temp);
                j += strlen(temp);
            } else {
                strcpy(&output[j], "$");
                j++;
                strcpy(&output[j], name);
                j += strlen(name);
            }
        } else {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

void pwd(char *arg) {
    printf("Built-in command executed: pwd\n");
    printf("Current state maintained in the command interpreter.\n");
}

void echo(char *arg) {
    printf("%s\n", arg);
}

void help(char *arg) {
    printf("Built-ins: echo, pwd, help\n");
}

void dispatch(char *cmd, char *arg) {
    CommandFunc table[] = {echo, pwd, help};
    char *names[] = {"echo", "pwd", "help"};

    for (int i = 0; i < 3; i++) {
        if (strcmp(cmd, names[i]) == 0) {
            table[i](arg);
            return;
        }
    }

    printf("Invalid command: %s\n", cmd);
}

int main() {
    char input[MAX];
    char expanded[MAX * 2];

    strcpy(input, "echo $MESSAGE");

    printf("Skill - 8: Variable Expansion and Built-in Dispatch\n");
    printf("--------------------------------------------------\n");
    printf("Input: %s\n", input);

    expand(input, expanded);
```

```
    printf("Expanded command: %s\n", expanded);

    char *cmd = strtok(expanded, " ");
    char *arg = strtok(NULL, "");

    if (cmd != NULL)
        dispatch(cmd, arg ? arg : "");

    return 0;
}
```

16. Program Explanation

16.1 Variable Table

The variable table stores sample names and values. The getValue() function searches the table and returns the value associated with a variable reference.

16.2 Expansion Function

The expand() function scans the input character by character. When a variable marker is detected, the variable name is extracted and replaced with its value. The function is recursive so that nested variable references can also be expanded.

16.3 Built-in Functions

The program contains sample built-ins such as echo, pwd, and help. Each built-in has a separate function and is called through the dispatch table when the corresponding command name is detected.

16.4 Dispatch Logic

The dispatch() function compares the command name against the built-in names. When a match is found, the corresponding function is called through a function pointer. If no match is found, an invalid-command message is displayed.

16.5 State Maintenance

Built-in commands are executed directly within the same process. This allows state maintained by the command interpreter to remain available after the built-in command finishes.

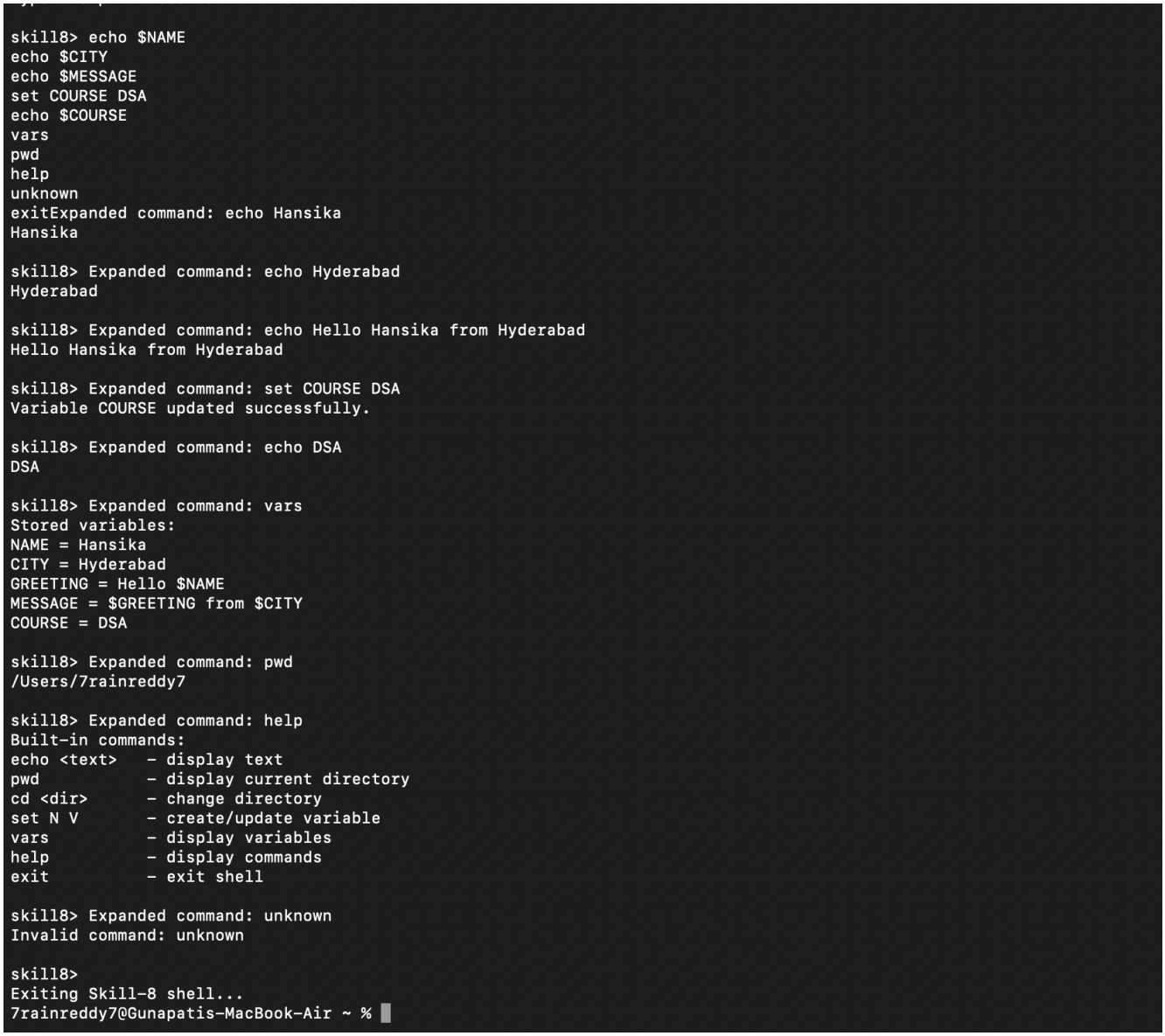
17. Sample Output

The following is an example of the expected program output:

```
Skill - 8: Variable Expansion and Built-in Dispatch
------------------------------------------------
Input: echo $MESSAGE
Expanded command: echo Hansika is from Hyderabad
Hansika is from Hyderabad
```

18. Output Screenshot

Paste the screenshot of the program output in the space provided below.



```
skill8> echo $NAME
echo $CITY
echo $MESSAGE
set COURSE DSA
echo $COURSE
vars
pwd
help
unknown
exitExpanded command: echo Hansika
Hansika

skill8> Expanded command: echo Hyderabad
Hyderabad

skill8> Expanded command: echo Hello Hansika from Hyderabad
Hello Hansika from Hyderabad

skill8> Expanded command: set COURSE DSA
Variable COURSE updated successfully.

skill8> Expanded command: echo DSA
DSA

skill8> Expanded command: vars
Stored variables:
NAME = Hansika
CITY = Hyderabad
GREETING = Hello $NAME
MESSAGE = $GREETING from $CITY
COURSE = DSA

skill8> Expanded command: pwd
/Users/7rainreddy7

skill8> Expanded command: help
Built-in commands:
echo <text>   - display text
pwd           - display current directory
cd <dir>      - change directory
set N V       - create/update variable
vars          - display variables
help          - display commands
exit          - exit shell

skill8> Expanded command: unknown
Invalid command: unknown

skill8>
Exiting Skill-8 shell...
7rainreddy7@Gunapatis-MacBook-Air ~ %
```

19. Testing

- Tested a directly referenced variable.
- Tested a variable whose value contains another variable.
- Tested an undefined variable.
- Verified that expanded tokens were updated.
- Tested a valid built-in command.
- Tested an invalid command.
- Verified that built-in execution maintained current-process state.

20. Test Cases

Test Case	Input	Expected Result
1	echo $NAME	Variable expands to Hansika.
2	echo $CITY	Variable expands to Hyderabad.
3	echo $MESSAGE	Nested variables expand to the complete message.
4	echo $UNKNOWN	Undefined variable is handled.
5	pwd	Built-in command is dispatched and executed in-process.
6	unknown	Invalid command message is displayed.

21. Observation

- Variable references were detected correctly.
- Stored variable values were substituted into the command.
- Nested variables were resolved successfully.
- Undefined variables were handled without stopping the parser.
- Updated tokens contained the expanded values.
- Built-in commands were identified through dispatch logic.
- Valid built-ins were executed in the current process.
- Invalid commands were reported correctly.
- The command interpreter state was maintained during built-in execution.

22. Result

Variable expansion and built-in command dispatch were successfully implemented. Variable references were detected and expanded, nested references were processed, undefined variables were handled, and the resulting tokens were updated. A dispatch table was used to identify and execute built-in commands in the current process while maintaining command-interpreter state.

23. Conclusion

The practical demonstrated how variable expansion can be integrated with command dispatch in a command interpreter. Correct token updating ensures that commands are executed with their intended values, while built-in dispatch allows state-changing commands to execute within the current process. The testing confirmed the required expansion and dispatch behavior.